JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class ForkJoinPool

java.lang.Object
    java.util.concurrent.AbstractExecutorService
        java.util.concurrent.ForkJoinPool

All Implemented Interfaces:

`Executor`, `ExecutorService`

```
public class ForkJoinPool
extends AbstractExecutorService
```

An `ExecutorService` for running `ForkJoinTask`s. A `ForkJoinPool` provides the entry point for submissions from non-`ForkJoinTask` clients, as well as management and monitoring operations.

A `ForkJoinPool` differs from other kinds of `ExecutorService` mainly by virtue of employing *work-stealing*: all threads in the pool attempt to find and execute tasks submitted to the pool and/or created by other active tasks (eventually blocking waiting for work if none exist). This enables efficient processing when most tasks spawn other subtasks (as do most `ForkJoinTask`s), as well as when many small tasks are submitted to the pool from external clients. Especially when setting *asyncMode* to true in constructors, `ForkJoinPool`s may also be appropriate for use with event-style tasks that are never joined.

A static `commonPool()` is available and appropriate for most applications. The common pool is used by any ForkJoinTask that is not explicitly submitted to a specified pool. Using the common pool normally reduces resource usage (its threads are slowly reclaimed during periods of non-use, and reinstated upon subsequent use).

For applications that require separate or custom pools, a `ForkJoinPool` may be constructed with a given target parallelism level; by default, equal to the number of available processors. The pool attempts to maintain enough active (or available) threads by dynamically adding, suspending, or resuming internal worker threads,

even if some tasks are stalled waiting to join others. However, no such adjustments are guaranteed in the face of blocked I/O or other unmanaged synchronization. The nested `ForkJoinPool.ManagedBlocker` interface enables extension of the kinds of synchronization accommodated.

In addition to execution and lifecycle control methods, this class provides status check methods (for example `getStealCount()`) that are intended to aid in developing, tuning, and monitoring fork/join applications. Also, method `toString()` returns indications of pool state in a convenient form for informal monitoring.

As is the case with other ExecutorServices, there are three main task execution methods summarized in the following table. These are designed to be used primarily by clients not already engaged in fork/join computations in the current pool. The main forms of these methods accept instances of `ForkJoinTask`, but overloaded forms also allow mixed execution of plain `Runnable`- or `Callable`-based activities as well. However, tasks that are already executing in a pool should normally instead use the within-computation forms listed in the table unless using async event-style tasks that are not usually joined, in which case there is little difference among choice of methods.

Summary of task execution methods

	Call from non-fork/join clients	**Call from within fork/join computations**
Arrange async execution	`execute(ForkJoinTask)`	`ForkJoinTask.fork()`
Await and obtain result	`invoke(ForkJoinTask)`	`ForkJoinTask.invoke()`
Arrange exec and obtain Future	`submit(ForkJoinTask)`	`ForkJoinTask.fork()` (ForkJoinTasks *are* Futures)

The common pool is by default constructed with default parameters, but these may be controlled by setting three system properties:

- `java.util.concurrent.ForkJoinPool.common.parallelism` - the parallelism level, a non-negative integer
- `java.util.concurrent.ForkJoinPool.common.threadFactory` - the class name of a `ForkJoinPool.ForkJoinWorkerThreadFactory`
- `java.util.concurrent.ForkJoinPool.common.exceptionHandler` - the class name of a `Thread.UncaughtExceptionHandler`

If a `SecurityManager` is present and no factory is specified, then the default pool uses a factory supplying threads that have no `Permissions` enabled. The system class loader is used to load these classes. Upon any error in establishing these settings, default parameters are used. It is possible to disable or limit the use of threads in the common pool by setting the parallelism property to zero, and/or using a factory that may return `null`. However doing so may cause unjoined tasks to never be executed.

Implementation notes: This implementation restricts the maximum number of running threads to 32767. Attempts to create pools with greater than the maximum number result in `IllegalArgumentException`.

This implementation rejects submitted tasks (that is, by throwing `RejectedExecutionException`) only when the pool is shut down or internal resources have been exhausted.

Since:

1.7

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static interface	**ForkJoinPool.ForkJoinWorkerThreadFactory**	Factory for creating new **ForkJoinWorkerThread**
static interface	**ForkJoinPool.ManagedBlocker**	Interface for extending managed parallelism for tasks running in **ForkJoinPool**s.

Field Summary

Fields

Modifier and Type	Field
static **ForkJoinPool.ForkJoinWorkerThreadFactory**	**defaultForkJoinWorkerThreadFacto**

Constructor Summary

Constructors

Constructor
ForkJoinPool()

ForkJoinPool(int parallelism)	
ForkJoinPool(int parallelism, **ForkJoinPool.ForkJoinWorkerThreadFactory** factory, T	

Method Summary

Concrete Methods

Modifier and Type	Method
boolean	**awaitQuiescence**(long timeout, **TimeUnit**
boolean	**awaitTermination**(long timeout, **TimeUnit**
static **ForkJoinPool**	**commonPool**()
protected int	**drainTasksTo**(**Collection**<? super **ForkJoi** >> c)
void	**execute**(**ForkJoinTask**<?> task)
void	**execute**(**Runnable** task)
int	**getActiveThreadCount**()

boolean	**getAsyncMode**()
static int	**getCommonPoolParallelism**()
ForkJoinPool.ForkJoinWorkerThreadFactory	**getFactory**()
int	**getParallelism**()
int	**getPoolSize**()
int	**getQueuedSubmissionCount**()
long	**getQueuedTaskCount**()
int	**getRunningThreadCount**()
long	**getStealCount**()
Thread.UncaughtExceptionHandler	**getUncaughtExceptionHandler**()

boolean	**hasQueuedSubmissions**()
<T> T	**invoke**(**ForkJoinTask**<T> task)
<T> **List**<**Future**<T>>	**invokeAll**(**Collection**<? extends **Callable**<T>> tasks)
boolean	**isQuiescent**()
boolean	**isShutdown**()
boolean	**isTerminated**()
boolean	**isTerminating**()
static void	**managedBlock**(**ForkJoinPool.ManagedBlocke**
protected <T> **RunnableFuture**<T>	**newTaskFor**(**Callable**<T> callable)
protected <T> **RunnableFuture**<T>	**newTaskFor**(**Runnable** runnable, T value)
protected **ForkJoinTask**<?>	**pollSubmission**()
void	**shutdown**()

List<Runnable>	shutdownNow()
<T> ForkJoinTask<T>	submit(Callable<T> task)
<T> ForkJoinTask<T>	submit(ForkJoinTask<T> task)
ForkJoinTask<?>	submit(Runnable task)
<T> ForkJoinTask<T>	submit(Runnable task, T result)
String	toString()

Methods inherited from class java.util.concurrent.AbstractExecutorService

invokeAll, invokeAny, invokeAny

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Detail

defaultForkJoinWorkerThreadFactory

```
public static final ForkJoinPool.ForkJoinWorkerThreadFactory defaultForkJoinWorke
```

Creates a new ForkJoinWorkerThread. This factory is used unless overridden in ForkJoinPool constructors.

Constructor Detail

ForkJoinPool

```
public ForkJoinPool()
```

Creates a `ForkJoinPool` with parallelism equal to `Runtime.availableProcessors()`, using the default thread factory, no UncaughtExceptionHandler, and non-async LIFO processing mode.

Throws:

`SecurityException` - if a security manager exists and the caller is not permitted to modify threads because it does not hold `RuntimePermission("modifyThread")`

ForkJoinPool

```
public ForkJoinPool(int parallelism)
```

Creates a `ForkJoinPool` with the indicated parallelism level, the default thread factory, no UncaughtExceptionHandler, and non-async LIFO processing mode.

Parameters:

`parallelism` - the parallelism level

Throws:

`IllegalArgumentException` - if parallelism less than or equal to zero, or greater than implementation limit

SecurityException - if a security manager exists and the caller is not permitted to modify threads because it does not hold RuntimePermission("modifyThread")

ForkJoinPool

```
public ForkJoinPool(int parallelism,
                    ForkJoinPool.ForkJoinWorkerThreadFactory factory,
                    Thread.UncaughtExceptionHandler handler,
                    boolean asyncMode)
```

Creates a ForkJoinPool with the given parameters.

Parameters:

parallelism - the parallelism level. For default value, use Runtime.availableProcessors().

factory - the factory for creating new threads. For default value, use defaultForkJoinWorkerThreadFactory.

handler - the handler for internal worker threads that terminate due to unrecoverable errors encountered while executing tasks. For default value, use null.

asyncMode - if true, establishes local first-in-first-out scheduling mode for forked tasks that are never joined. This mode may be more appropriate than default locally stack-based mode in applications in which worker threads only process event-style asynchronous tasks. For default value, use false.

Throws:

IllegalArgumentException - if parallelism less than or equal to zero, or greater than implementation limit

NullPointerException - if the factory is null

SecurityException - if a security manager exists and the caller is not permitted to modify threads because it does not hold RuntimePermission("modifyThread")

Method Detail

commonPool

```
public static ForkJoinPool commonPool()
```

Returns the common pool instance. This pool is statically constructed; its run state is unaffected by attempts to shutdown() or shutdownNow(). However

this pool and any ongoing processing are automatically terminated upon program System.exit(int). Any program that relies on asynchronous task processing to complete before program termination should invoke commonPool().awaitQuiescence, before exit.

Returns:

the common pool instance

Since:

1.8

invoke

public <T> T invoke(ForkJoinTask<T> task)

Performs the given task, returning its result upon completion. If the computation encounters an unchecked Exception or Error, it is rethrown as the outcome of this invocation. Rethrown exceptions behave in the same way as regular exceptions, but, when possible, contain stack traces (as displayed for example using ex.printStackTrace()) of both the current thread as well as the thread actually encountering the exception; minimally only the latter.

Type Parameters:

T - the type of the task's result

Parameters:

task - the task

Returns:

the task's result

Throws:

NullPointerException - if the task is null

RejectedExecutionException - if the task cannot be scheduled for execution

execute

```
public void execute(ForkJoinTask<?> task)
```

Arranges for (asynchronous) execution of the given task.

Parameters:

`task` - the task

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

execute

```
public void execute(Runnable task)
```

Description copied from interface: `Executor`

Executes the given command at some time in the future. The command may execute in a new thread, in a pooled thread, or in the calling thread, at the discretion of the `Executor` implementation.

Parameters:

`task` - the runnable task

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

submit

```
public <T> ForkJoinTask<T> submit(ForkJoinTask<T> task)
```

Submits a ForkJoinTask for execution.

Type Parameters:

`T` - the type of the task's result

Parameters:

`task` - the task to submit

Returns:

the task

Throws:

`NullPointerException` - if the task is null

RejectedExecutionException - if the task cannot be scheduled for execution

submit

```
public <T> ForkJoinTask<T> submit(Callable<T> task)
```

Description copied from interface: ExecutorService

Submits a value-returning task for execution and returns a Future representing the pending results of the task. The Future's `get` method will return the task's result upon successful completion.

If you would like to immediately block waiting for a task, you can use constructions of the form `result = exec.submit(aCallable).get();`

Note: The `Executors` class includes a set of methods that can convert some other common closure-like objects, for example, `PrivilegedAction` to `Callable` form so they can be submitted.

Specified by:

`submit` in interface `ExecutorService`

Overrides:

`submit` in class `AbstractExecutorService`

Type Parameters:

`T` - the type of the task's result

Parameters:

`task` - the task to submit

Returns:

a Future representing pending completion of the task

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

submit

```
public <T> ForkJoinTask<T> submit(Runnable task,
                                  T result)
```

Description copied from interface: ExecutorService

Submits a Runnable task for execution and returns a Future representing that task. The Future's `get` method will return the given result upon successful completion.

Specified by:

`submit` in interface `ExecutorService`

Overrides:

`submit` in class `AbstractExecutorService`

Type Parameters:

`T` - the type of the result

Parameters:

`task` - the task to submit

`result` - the result to return

Returns:

a Future representing pending completion of the task

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

submit

`public ForkJoinTask<?> submit(Runnable task)`

Description copied from interface: `ExecutorService`

Submits a Runnable task for execution and returns a Future representing that task. The Future's `get` method will return `null` upon *successful* completion.

Specified by:

`submit` in interface `ExecutorService`

Overrides:

`submit` in class `AbstractExecutorService`

Parameters:

`task` - the task to submit

Returns:

a Future representing pending completion of the task

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

invokeAll

```
public <T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)
```

Description copied from interface: ExecutorService

Executes the given tasks, returning a list of Futures holding their status and results when all complete. `Future.isDone()` is `true` for each element of the returned list. Note that a *completed* task could have terminated either normally or by throwing an exception. The results of this method are undefined if the given collection is modified while this operation is in progress.

Specified by:

`invokeAll` in interface `ExecutorService`

Overrides:

`invokeAll` in class `AbstractExecutorService`

Type Parameters:

`T` - the type of the values returned from the tasks

Parameters:

`tasks` - the collection of tasks

Returns:

a list of Futures representing the tasks, in the same sequential order as produced by the iterator for the given task list, each of which has completed

Throws:

`NullPointerException` - if tasks or any of its elements are null

`RejectedExecutionException` - if any task cannot be scheduled for execution

getFactory

```
public ForkJoinPool.ForkJoinWorkerThreadFactory getFactory()
```

Returns the factory used for constructing new workers.

Returns:

the factory used for constructing new workers

getUncaughtExceptionHandler

```
public Thread.UncaughtExceptionHandler getUncaughtExceptionHandler()
```

Returns the handler for internal worker threads that terminate due to unrecoverable errors encountered while executing tasks.

Returns:

`the handler, or null if none`

getParallelism

```
public int getParallelism()
```

Returns the targeted parallelism level of this pool.

Returns:

`the targeted parallelism level of this pool`

getCommonPoolParallelism

```
public static int getCommonPoolParallelism()
```

Returns the targeted parallelism level of the common pool.

Returns:

`the targeted parallelism level of the common pool`

Since:

`1.8`

getPoolSize

```
public int getPoolSize()
```

Returns the number of worker threads that have started but not yet terminated. The result returned by this method may differ from `getParallelism()` when threads are created to maintain parallelism when others are cooperatively blocked.

Returns:

`the number of worker threads`

getAsyncMode

```
public boolean getAsyncMode()
```

Returns `true` if this pool uses local first-in-first-out scheduling mode for forked tasks that are never joined.

Returns:

`true if this pool uses async mode`

getRunningThreadCount

```
public int getRunningThreadCount()
```

Returns an estimate of the number of worker threads that are not blocked waiting to join tasks or for other managed synchronization. This method may overestimate the number of running threads.

Returns:

`the number of worker threads`

getActiveThreadCount

```
public int getActiveThreadCount()
```

Returns an estimate of the number of threads that are currently stealing or executing tasks. This method may overestimate the number of active threads.

Returns:

`the number of active threads`

isQuiescent

```
public boolean isQuiescent()
```

Returns `true` if all worker threads are currently idle. An idle worker is one that cannot obtain a task to execute because none are available to steal from other threads, and there are no pending submissions to the pool. This method is conservative; it might not return `true` immediately upon idleness of all threads, but will eventually become true if threads remain inactive.

Returns:

`true if all threads are currently idle`

getStealCount

```
public long getStealCount()
```

Returns an estimate of the total number of tasks stolen from one thread's work queue by another. The reported value underestimates the actual total number of steals when the pool is not quiescent. This value may be useful for monitoring and tuning fork/join programs: in general, steal counts should be

high enough to keep threads busy, but low enough to avoid overhead and contention across threads.

Returns:

`the number of steals`

getQueuedTaskCount

```
public long getQueuedTaskCount()
```

Returns an estimate of the total number of tasks currently held in queues by worker threads (but not including tasks submitted to the pool that have not begun executing). This value is only an approximation, obtained by iterating across all threads in the pool. This method may be useful for tuning task granularities.

Returns:

`the number of queued tasks`

getQueuedSubmissionCount

```
public int getQueuedSubmissionCount()
```

Returns an estimate of the number of tasks submitted to this pool that have not yet begun executing. This method may take time proportional to the number of submissions.

Returns:

`the number of queued submissions`

hasQueuedSubmissions

```
public boolean hasQueuedSubmissions()
```

Returns `true` if there are any tasks submitted to this pool that have not yet begun executing.

Returns:

`true if there are any queued submissions`

pollSubmission

```
protected ForkJoinTask<?> pollSubmission()
```

Removes and returns the next unexecuted submission if one is available. This method may be useful in extensions to this class that re-assign work in systems with multiple pools.

Returns:

the next submission, or null if none

drainTasksTo

```
protected int drainTasksTo(Collection<? super ForkJoinTask<?>> c)
```

Removes all available unexecuted submitted and forked tasks from scheduling queues and adds them to the given collection, without altering their execution status. These may include artificially generated or wrapped tasks. This method is designed to be invoked only when the pool is known to be quiescent. Invocations at other times may not remove all tasks. A failure encountered while attempting to add elements to collection `c` may result in elements being in neither, either or both collections when the associated exception is thrown. The behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Parameters:

c - the collection to transfer elements into

Returns:

the number of elements transferred

toString

```
public String toString()
```

Returns a string identifying this pool, as well as its state, including indications of run state, parallelism level, and worker and task counts.

Overrides:

toString in class Object

Returns:

a string identifying this pool, as well as its state

shutdown

```
public void shutdown()
```

Possibly initiates an orderly shutdown in which previously submitted tasks are executed, but no new tasks will be accepted. Invocation has no effect on execution state if this is the commonPool(), and no additional effect if already

shut down. Tasks that are in the process of being submitted concurrently during the course of this method may or may not be rejected.

Throws:

`SecurityException - if a security manager exists and the caller is not permitted to modify threads because it does not hold RuntimePermission("modifyThread")`

shutdownNow

```
public List<Runnable> shutdownNow()
```

Possibly attempts to cancel and/or stop all tasks, and reject all subsequently submitted tasks. Invocation has no effect on execution state if this is the `commonPool()`, and no additional effect if already shut down. Otherwise, tasks that are in the process of being submitted or executed concurrently during the course of this method may or may not be rejected. This method cancels both existing and unexecuted tasks, in order to permit termination in the presence of task dependencies. So the method always returns an empty list (unlike the case for some other Executors).

Returns:

`an empty list`

Throws:

`SecurityException - if a security manager exists and the caller is not permitted to modify threads because it does not hold RuntimePermission("modifyThread")`

isTerminated

```
public boolean isTerminated()
```

Returns `true` if all tasks have completed following shut down.

Returns:

`true if all tasks have completed following shut down`

isTerminating

```
public boolean isTerminating()
```

Returns `true` if the process of termination has commenced but not yet completed. This method may be useful for debugging. A return of `true` reported a sufficient period after shutdown may indicate that submitted tasks have ignored or suppressed interruption, or are waiting for I/O, causing this executor not to properly terminate. (See the advisory notes for class

ForkJoinTask stating that tasks should not normally entail blocking operations. But if they do, they must abort them on interrupt.)

Returns:

`true if terminating but not yet terminated`

isShutdown

```
public boolean isShutdown()
```

Returns `true` if this pool has been shut down.

Returns:

`true if this pool has been shut down`

awaitTermination

```
public boolean awaitTermination(long timeout,
                                TimeUnit unit)
                         throws InterruptedException
```

Blocks until all tasks have completed execution after a shutdown request, or the timeout occurs, or the current thread is interrupted, whichever happens first. Because the `commonPool()` never terminates until program shutdown, when applied to the common pool, this method is equivalent to `awaitQuiescence(long, TimeUnit)` but always returns `false`.

Parameters:

`timeout - the maximum time to wait`

`unit - the time unit of the timeout argument`

Returns:

`true if this executor terminated and false if the timeout elapsed before termination`

Throws:

`InterruptedException - if interrupted while waiting`

awaitQuiescence

```
public boolean awaitQuiescence(long timeout,
                               TimeUnit unit)
```

If called by a ForkJoinTask operating in this pool, equivalent in effect to `ForkJoinTask.helpQuiesce()`. Otherwise, waits and/or attempts to assist performing tasks until this pool `isQuiescent()` or the indicated timeout elapses.

Parameters:

`timeout - the maximum time to wait`

`unit - the time unit of the timeout argument`

Returns:

`true if quiescent; false if the timeout elapsed.`

managedBlock

```
public static void managedBlock(ForkJoinPool.ManagedBlocker blocker)
                         throws InterruptedException
```

Runs the given possibly blocking task. When running in a ForkJoinPool, this method possibly arranges for a spare thread to be activated if necessary to ensure sufficient parallelism while the current thread is blocked in `blocker.block()`.

This method repeatedly calls `blocker.isReleasable()` and `blocker.block()` until either method returns `true`. Every call to `blocker.block()` is preceded by a call to `blocker.isReleasable()` that returned `false`.

If not running in a ForkJoinPool, this method is behaviorally equivalent to

```
while (!blocker.isReleasable())
  if (blocker.block())
    break;
```

If running in a ForkJoinPool, the pool may first be expanded to ensure sufficient parallelism available during the call to `blocker.block()`.

Parameters:

`blocker - the blocker task`

Throws:

`InterruptedException - if blocker.block() did so`

newTaskFor

```
protected <T> RunnableFuture<T> newTaskFor(Runnable runnable,
                                           T value)
```

Description copied from class: AbstractExecutorService

Returns a `RunnableFuture` for the given runnable and default value.

Overrides:

`newTaskFor in class AbstractExecutorService`

Type Parameters:

`T - the type of the given value`

Parameters:

`runnable - the runnable task being wrapped`

`value - the default value for the returned future`

Returns:

`a RunnableFuture which, when run, will run the underlying runnable and which, as a Future, will yield the given value as its result and provide for cancellation of the underlying task`

newTaskFor

`protected <T> RunnableFuture<T> newTaskFor(Callable<T> callable)`

Description copied from class: AbstractExecutorService

Returns a RunnableFuture for the given callable task.

Overrides:

`newTaskFor in class AbstractExecutorService`

Type Parameters:

`T - the type of the callable's result`

Parameters:

`callable - the callable task being wrapped`

Returns:

`a RunnableFuture which, when run, will call the underlying callable and which, as a Future, will yield the callable's result as its result and provide for cancellation of the underlying task`

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms,

workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.